

Documentação Arquitetural e Defesa Técnica

Plataforma de Microsserviços e DevOps

Paulo Vinícius Holanda Gomes

23 de junho de 2026

Conteúdo

1	Arquitetura do Sistema e Decisões Técnicas	2
1.1	Isolamento de Domínios e Bancos de Dados Independentes	2
1.2	API Gateway e Roteamento	2
1.3	Orquestração e Alta Disponibilidade	2
1.4	Observabilidade e Monitoramento	3
2	Diagrama Arquitetural e Topologia de Rede	3
3	Fluxo de Integração e Entrega Contínuas (CI/CD)	3
3.1	Integração Contínua e Testes Isolados	3
3.2	Qualidade de Código e Segurança	4
3.3	Entrega e Validação <i>End-to-End</i> (Smoke Tests)	4
4	Instruções de Deploy e Operação	4
4.1	Pré-requisitos e Inicialização	5
4.2	Execução do Deploy Automatizado	5
4.3	Validação e Procedimento de Rollback	5
4.4	Encerramento da Infraestrutura	6
5	Testes de Resiliência e Operação Final	6
5.1	Maturidade REST e HATEOAS	6
5.2	Resiliência e Autocura (Self-Healing)	6
5.3	Monitoramento Contínuo com Grafana	7

1 Arquitetura do Sistema e Decisões Técnicas

Nesta seção, detalhamos a estrutura da plataforma distribuída e as escolhas de engenharia feitas para garantir escalabilidade, segurança e estabilidade.

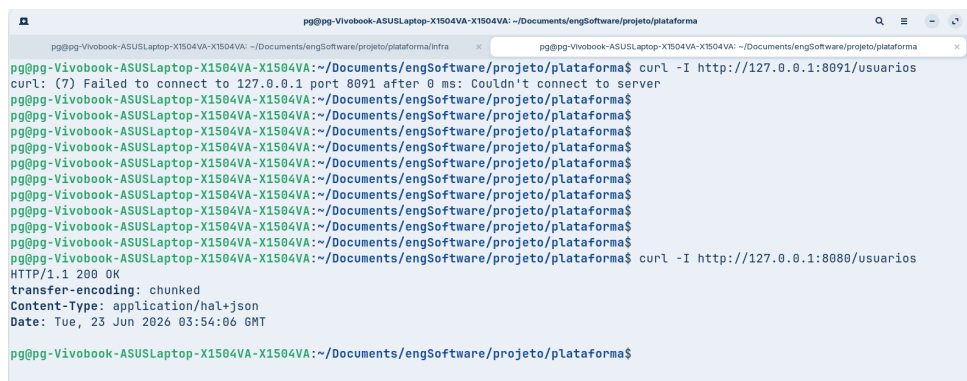
1.1 Isolamento de Domínios e Bancos de Dados Independentes

O sistema foi dividido em três domínios de negócio: Usuários (`auth-service`), Disciplinas (`academic-service`) e Atividades (`assignment-service`). Para evitar que a queda de um banco afete o sistema inteiro, cada microserviço possui sua própria instância de banco de dados PostgreSQL. A persistência física dos dados é garantida por volumes do Docker. Como os dados são isolados, as consultas entre os serviços ocorrem exclusivamente por requisições HTTP na rede interna (via `RestTemplate`).

1.2 API Gateway e Roteamento

O Spring Cloud Gateway atua como o único ponto de entrada do sistema (porta 8080). As portas individuais dos microserviços não são expostas publicamente. O Gateway recebe o tráfego externo e o direciona para o contêiner correto. Ele também funciona como um proxy reverso que mantém os cabeçalhos originais das requisições intactos. Isso é essencial para que os links de navegação da API (padrão HATEOAS) sempre mostrem o endereço público correto, ocultando completamente a rede interna.

Para comprovar a restrição do tráfego Norte-Sul, a Figura 1 exibe uma tentativa de acesso direto à porta de um microserviço (recusada) contraposta ao acesso bem-sucedido roteado exclusivamente pela porta 8080 do Gateway.



```
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$ curl -I http://127.0.0.1:8091/usuarios
curl: (7) Failed to connect to 127.0.0.1 port 8091 after 0 ms: Couldn't connect to server
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$ curl -I http://127.0.0.1:8080/usuarios
HTTP/1.1 200 OK
transfer-encoding: chunked
Content-Type: application/hal+json
Date: Tue, 23 Jun 2026 03:54:06 GMT
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$
```

Figura 1: Testes de conectividade evidenciando a proteção das portas internas.

1.3 Orquestração e Alta Disponibilidade

O gerenciamento da infraestrutura é feito pelo Docker Swarm. Todos os serviços se comunicam por meio de uma rede virtual isolada. Para manter o sistema sempre disponível, configuramos duas cópias simultâneas (`replicas: 2`) para cada microserviço. O próprio Swarm balanceia a carga entre essas cópias e recria contêineres automaticamente caso algum falhe. O arquivo de orquestração (`docker-stack.yml`) também define limites máximos de memória para cada serviço, prevenindo sobrecargas no servidor.

A Figura 2 ilustra o *cluster* orquestrado pelo Docker Swarm, atestando a configuração de alta disponibilidade com duas réplicas (2/2) para os microserviços de negócio.

```
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma
pg@pg-Vivobook-ASUSLaptop-X1504VA-X1504VA: ~/Documents/engSoftware/projeto/plataforma$ docker service ls
ID                NAME                                MODE                REPLICAS  IMAGE                                PORTS
d31nbtjb22se      plataforma-devops_academic-service  replicated          2/2        pgomes61/academic-service:latest
leu91glnioc0      plataforma-devops_api-gateway        replicated          1/1        pgomes61/api-gateway:latest        *:8080->8080/tcp
h58m5grj8aa0      plataforma-devops_assignment-service  replicated          2/2        pgomes61/assignment-service:latest
ipsrcvxxreez      plataforma-devops_auth-service       replicated          2/2        pgomes61/auth-service:latest
y5mg72s28zz3      plataforma-devops_db_academic         replicated          1/1        postgres:15-alpine
0vzz84iu43fg      plataforma-devops_db_assignment       replicated          1/1        postgres:15-alpine
yed6f09tjta8      plataforma-devops_db_auth             replicated          1/1        postgres:15-alpine
w08aa6lxbecs      plataforma-devops_grafana             replicated          1/1        grafana/grafana:latest             *:3000->3000/tcp
uv2h4vetw2pk      plataforma-devops_prometheus          replicated          1/1        prom/prometheus:latest
```

Figura 2: Status dos serviços em operação via comando `docker service ls`.

1.4 Observabilidade e Monitoramento

A saúde e o desempenho das aplicações são monitorados em tempo real. O Spring Boot fornece as métricas, e o Prometheus atua nos bastidores da rede interna (sem acesso à internet, por segurança) para coletar esses dados. Os gráficos são visualizados no Grafana (porta 3000). Todo o ambiente de monitoramento já sobe pré-configurado e com o acesso anônimo ativado, permitindo visualizar os painéis imediatamente, sem exigir telas de login.

2 Diagrama Arquitetural e Topologia de Rede

A Figura 3 ilustra a topologia da plataforma operando sob a orquestração do Docker Swarm. O diagrama mapeia o fluxo de comunicação do sistema, destacando o Ponto Único de Entrada (API Gateway) gerenciando o Tráfego Norte-Sul na porta 8080, e a comunicação Leste-Oeste entre os microsserviços isolados na rede *overlay*. Adicionalmente, a representação evidencia o isolamento da persistência de dados (*Database-per-Service*) e a topologia de telemetria invisível para o ambiente externo.

3 Fluxo de Integração e Entrega Contínuas (CI/CD)

O fluxo de versionamento do projeto adotou a abordagem *Trunk-Based Development*. Toda a automação é gerenciada pelo GitHub Actions, onde a esteira garante que qualquer alteração no código seja testada, analisada e validada em um ambiente real antes de ser liberada.

3.1 Integração Contínua e Testes Isolados

A fase de CI compila o código e executa os testes automatizados. Para garantir a confiabilidade da persistência de dados, utilizamos a biblioteca Testcontainers. Ela sobe instâncias temporárias do PostgreSQL no momento da execução, permitindo que os testes de integração validem as operações em um banco de dados real. Após os testes, o contêiner é destruído automaticamente.

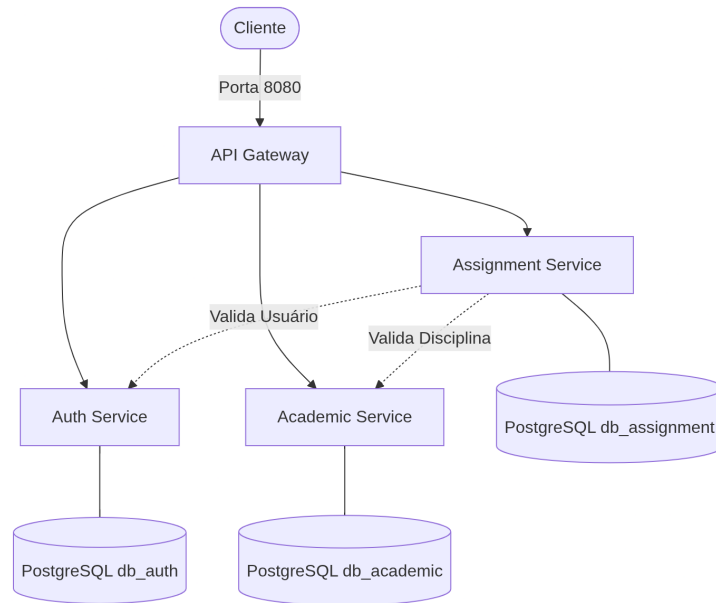


Figura 3: Arquitetura de Microserviços e Topologia no Docker Swarm.

3.2 Qualidade de Código e Segurança

Ainda na fase de integração, o sistema passa por duas barreiras automatizadas:

- **Análise Estática (PMD):** Faz uma varredura no código-fonte em busca de falhas lógicas, más práticas e problemas de padronização.
- **Varredura de Segurança (Trivy):** Analisa os arquivos do projeto em busca de vulnerabilidades e falhas de segurança conhecidas (CVEs). A esteira é configurada para falhar e bloquear o processo caso encontre riscos de nível crítico.

3.3 Entrega e Validação *End-to-End* (Smoke Tests)

Após a aprovação na fase de CI, as imagens Docker são construídas e enviadas para o Docker Hub. Em seguida, a esteira inicializa um *cluster* Docker Swarm temporário no próprio servidor do GitHub e realiza o deploy de toda a infraestrutura. Por fim, um script de *Smoke Test* executa tentativas de conexão contínuas contra o API Gateway para atestar se o roteamento para os três microserviços está funcionando corretamente. Se o teste retornar sucesso, a versão é considerada estável e apta para produção.

A Figura 4 demonstra a execução bem-sucedida da esteira no GitHub Actions, englobando a compilação, a análise estática e a realização dos *Smoke Tests* contra o API Gateway.

4 Instruções de Deploy e Operação

Esta seção descreve os comandos exatos para inicializar a infraestrutura, rodar a plataforma no ambiente de produção e encerrar a execução.

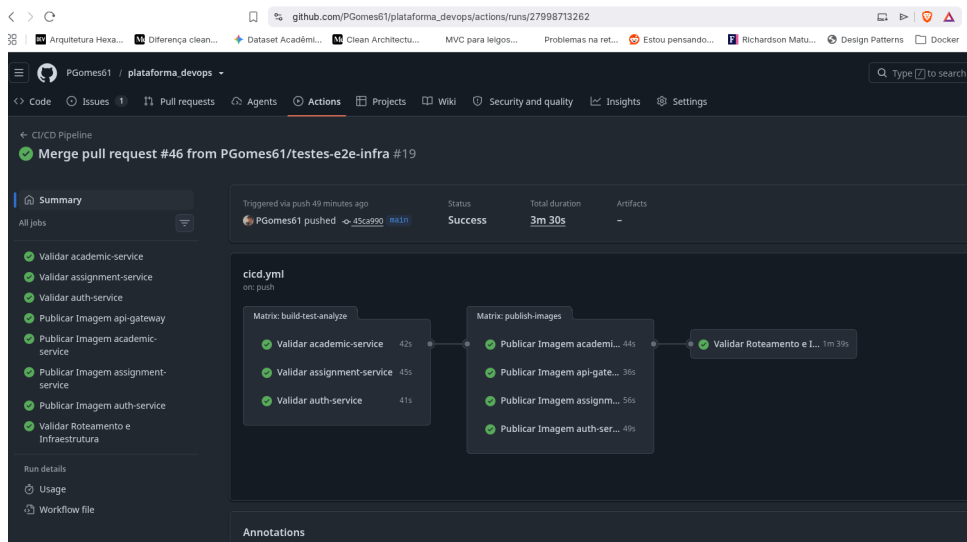


Figura 4: Execução com sucesso do pipeline CI/CD.

4.1 Pré-requisitos e Inicialização

O servidor deve possuir o Docker instalado. A ativação do modo orquestrador é o primeiro passo obrigatório e é realizada com o comando básico na raiz do terminal:

```
1 docker swarm init
```

4.2 Execução do Deploy Automatizado

Todos os arquivos de configuração e provisionamento estão isolados e organizados dentro do diretório `infra/`. A partir da raiz do projeto clonado, o processo de subida do ambiente exige a seguinte sequência de comandos:

```
1 # 1. Acesse o diretorio de infraestrutura
2 cd infra
3
4 # 2. Garanta a permissao de execucao dos scripts
5 chmod +x deploy.sh smoke-test.sh
6
7 # 3. Execute o deploy automatizado
8 ./deploy.sh
```

O script realiza o ajuste de permissões para os diretórios locais do Grafana, baixa as imagens mais recentes do registro e aplica a orquestração via `docker stack deploy`, garantindo que o sistema suba sem atritos.

4.3 Validação e Procedimento de Rollback

Para atestar que o roteamento interno e os contêineres subiram corretamente, utilize o script de validação de conectividade ainda dentro da pasta `infra/`:

```
1 ./smoke-test.sh http://127.0.0.1:8080
```

A infraestrutura conta com resiliência nativa. Caso uma atualização introduza instabilidade em produção, é possível retornar instantaneamente para a versão funcional anterior através do comando:

```
1 docker service update --rollback plataforma-devops_<service_name>
```

4.4 Encerramento da Infraestrutura

Para interromper a execução do sistema e limpar os recursos, basta remover a *stack* (nomeada como *plataforma-devops* no provisionamento). Esse comando derruba os serviços e as redes virtuais em segurança:

```
1 docker stack rm plataforma-devops
```

Caso seja necessário desativar completamente o modo orquestrador do Docker na máquina hospedeira após o uso, executa-se:

```
1 docker swarm leave --force
```

5 Testes de Resiliência e Operação Final

Após a finalização do *deploy*, a plataforma foi submetida a testes práticos para atestar sua resiliência, maturidade e capacidade de observabilidade no ambiente de produção.

5.1 Maturidade REST e HATEOAS

A Figura 5 apresenta uma requisição realizada via terminal utilizando a ferramenta *cURL*. É possível observar o retorno do nó `_links`, atestando o Nível 3 de Richardson e a atuação do *proxy* reverso, que expõe URLs apontando para o Gateway (`localhost:8080`), ocultando a infraestrutura interna.



Figura 5: Resposta da API exibindo os links dinâmicos do HATEOAS.

5.2 Resiliência e Autocura (Self-Healing)

A Figura 6 ilustra a recuperação autônoma do Swarm. Após o encerramento forçado de um contêiner, o orquestrador detecta a falha e provisiona imediatamente uma nova instância para manter o estado desejado da aplicação.

The terminal shows the execution of `docker ps` and `docker kill` commands. Below is a table representing the state of the containers shown in the terminal output.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0ba43bd716e8	pgones61/auth-service:latest	"java -jar app.jar"	53 seconds ago	Up 47 seconds	8088/tcp	platforma-devops_auth-service.2.pv6f3aoavvg0mzn3rfojc7y93b
c7f5c579207b	prna/prometheus:latest	"/bin/prometheus -c..."	36 hours ago	Up 36 hours	9090/tcp	platforma-devops_prometheus.1.xscoqbfmuh4en36frabvzj3y
872eb9a8dccc	pgones61/academic-service:latest	"java -jar app.jar"	36 hours ago	Up 36 hours	8088/tcp	platforma-devops_academic-service.2.4pyd812v91o7w2og81faypj
28fb31b4bc8f	pgones61/academic-service:latest	"java -jar app.jar"	36 hours ago	Up 36 hours	8088/tcp	platforma-devops_academic-service.1.xp5tyz54v2v33ycqqd0dpu
073b393f9657	pgones61/assignment-service:latest	"java -jar app.jar"	36 hours ago	Up 36 hours	8088/tcp	platforma-devops_assignment-service.2.up2ubv85denzu8u8a9f5v7w
ecbb655a8e6c	pgones61/assignment-service:latest	"java -jar app.jar"	36 hours ago	Up 36 hours	8088/tcp	platforma-devops_assignment-service.1.12u7bhg02dftatf7q7nk3gdf9
2aa2a896be98	postgres:15-alpine	"docker-entrypoint.s..."	36 hours ago	Up 36 hours	5432/tcp	platforma-devops_db_academic.1.i9uyhsvvc7xlp7nxxmhh9ontf
771f4c4e65bc	postgres:15-alpine	"docker-entrypoint.s..."	36 hours ago	Up 36 hours	5432/tcp	platforma-devops_db_assignment.1.jeojseu865uss8m951fvn8fs
2152aef68e93	grafana/grafana:latest	"/run.sh"	36 hours ago	Up 36 hours	3000/tcp	platforma-devops_grafana.1.fpb6iajg3cfwz21kh56ynzaw
5fb345f75fea	pgones61/api-gateway:latest	"java -jar app.jar"	36 hours ago	Up 36 hours	8088/tcp	platforma-devops_api-gateway.1.8xwslq3l7co8vab6kynlcnuvx
97ca15b18dae	postgres:15-alpine	"docker-entrypoint.s..."	36 hours ago	Up 36 hours	5432/tcp	platforma-devops_db_auth.1.r8p2hbptut1ljvf34necrx05
630c70ed888f	pgones61/auth-service:latest	"java -jar app.jar"	36 hours ago	Up 36 hours	8088/tcp	platforma-devops_auth-service.1.1b0dt8x43pwxjzccnt5byf0

ID	NAME	IMAGE	NODE	service ps	platforma-devops_auth-service	DESIRED STATE	CURRENT STATE	ERROR
wykwpxjaye6	platforma-devops_auth-service.1	pgones61/auth-service:latest	pg-Vivobook-ASUSLaptop-X1504VA-X1504VA	Running	Running	Running	2 seconds ago	
1b0dt8x43px	platforma-devops_auth-service.1	pgones61/auth-service:latest	pg-Vivobook-ASUSLaptop-X1504VA-X1504VA	Shutdown	Failed	8 seconds ago		"task: non-zero exi
pv6f3aoavvg0	platforma-devops_auth-service.2	pgones61/auth-service:latest	pg-Vivobook-ASUSLaptop-X1504VA-X1504VA	Running	Running	about a minute ago		
1b0dt8x43px	platforma-devops_auth-service.2	pgones61/auth-service:latest	pg-Vivobook-ASUSLaptop-X1504VA-X1504VA	Shutdown	Failed	about a minute ago		"task: non-zero exi

Figura 6: Substituição automática de um contêiner com falha (*Self-Healing*).

5.3 Monitoramento Contínuo com Grafana

Por fim, a Figura 7 comprova a integração da telemetria, exibindo os dados de consumo e tráfego da rede *overlay* consolidados nos painéis provisionados do Grafana.

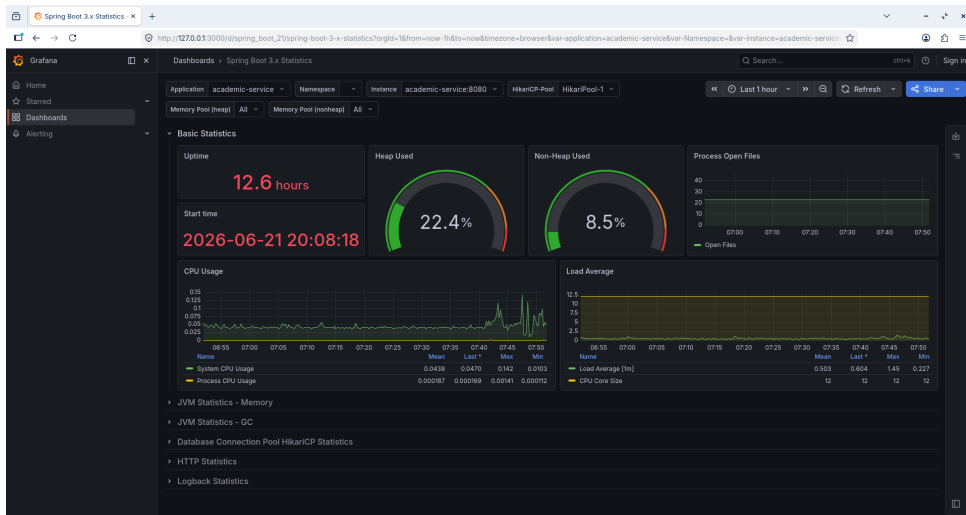


Figura 7: Painel do Grafana preenchido com métricas da plataforma em tempo real.